A UVM-based Verification Approach for MIPI DSI Low-Level Protocol layer

Ahmed Mohamed Ali
Computer and Systems Engineering Department
Ain Shams University
Cairo, Egypt
ahmed.aly.private@gmail.com

Ahmed Shalaby
Computer Science Department
Benha University
Benha, Egypt
ahmed.shalaby@fci.bu.edu.eg

Sherif Saif
Computers and Systems Department
Electronics Research Institute
Cairo, Egypt
sherif.saif@eri.sci.eg

Mohamed Taher
Computer and Systems Engineering Department
Ain Shams University
Cairo, Egypt
mohamed.taher@eng.asu.edu.eg

2022 International Conference on Microelectronics (ICM) | 978-1-6654-9324-6/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICM56065.2022.10005400

***Abstract*— Display Serial Interface (DSI) is a high-speed serial interface standard. It supports display and touch screens in mobile devices such as smartphones, laptops, tablets, and other platforms. DSI describes several layers that define the detailed interconnect between a host processor and a peripheral device in a mobile system. The targeted DSI protocol layer is the low-level layer in the standard which is responsible for bytes organization, error-checking information addition, and packet formulation. In this paper, we propose a constrained random Coverage-Driven Verification approach (CDV) for the DSI low-level protocol layer using Universal Verification Methodology (UVM). This approach could be the basis of a standardized Verification Intellectual Property (VIP) to test and verify the standardized DSI layer. We provide a well-established base of a reusable and scalable UVM environment that can verify the DSI protocol layer using various techniques such as error injection mechanism, System Verilog Assertions (SVA), and direct UVM sequences aim to cover the different real-life scenarios between the host processor and the peripheral device. Our results show that we can detect all inserted errors to assure the functionality of the DSI low-level protocol layer. Different real-life scenarios between the host processor and the peripheral device are covered with 100% functional coverage and 93% code coverage.**

Keywords—Display Serial Interface (DSI), Mobile Industry Processor Interface (MIPI), Universal Verification Methodology (UVM), Verification Intellectual Property (VIP).

I. INTRODUCTION

Mobile Industry Processor Interface Alliance (MIPI) is a global collaborative organization that serves the mobile and mobile-influenced industry. It was founded in 2003 by Advanced RISC Machine (ARM), ST Micro-electronics, Nokia, and Texas Instruments to consolidate standardized high-performance hardware and software interfaces [1, 10]. These interfaces serve mobile and mobile-influenced industries such as smartphones, laptops, tablets, and wearable devices [1]. The standard specifications should meet the operating conditions constraints of mobile devices such as high-bandwidth performance, low power consumption, and low electromagnetic interference (EMI) [12]. Multimedia Display Serial Interface (DSI) is one of the most well-known specifications provided by MIPI [10]. It is a versatile high-speed serial interface that supports the display and touch Multimedia sector. It has been released in 2004 and continued to be updated to meet the demand for faster frame rates, higher resolution images, and greater color depth. DSI standard [2, 9] follows the Open System Interconnection model (OSI) in defining a set of layers to describe the detailed interconnection between the host application processor and the peripheral device. Four layers are defined which are: (1) PHY layer, (2) lane management layer, (3) low-level protocol layer, and (4) application layer. Fig. 1 demonstrates the layers' definition and communications in between.

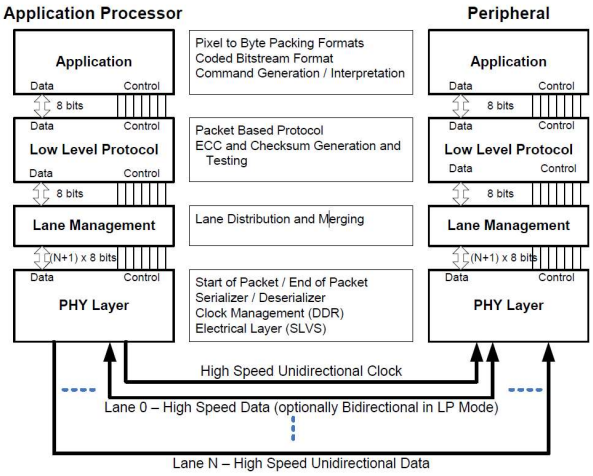

Fig. 1. DSI layers' definitions [2].

DSI low-level protocol layer - layer 3- is the targeted layer to be verified. It specifies the packet structure and the header information. On the transmitter, the header and error-checking information are appended to the transmitted packet. On the receiver, the header is stripped off, and error-checking information is used to verify the integrity of the incoming data [2]. The mentioned layer is the ***digital* module** of the DSI. One of the challenges of the DSI implementation is the verification of this layer [5]. In [6], authors proposed ordinary testbenches and integrated hardware emulation testing to verify this layer.

This paper proposes a scalable, reusable UVM-based verification environment that could be a foundation of a standardized Verification Intellectual Property (VIP) for the standardized ***DSI low-level protocol layer***. The rest of the paper is organized as follows. Section II presents our implementation of the low-level protocol layer. Section III discusses the proposed verification plan. While section IV details the verification environment. Section V shows the results. Finally, Section VI concludes the paper.

II. DSI PROTOCOL LAYER DESIGN IMPLEMENTATION

In this section, we demonstrate our implementation of the DSI protocol layer that follows the standard in terms of performance and functionality. Fig. 2 shows the block diagram of the proposed design structure. The main building blocks are as follows:

A. *SPI Slave:* Serial Peripheral Interface (SPI) is a standardized synchronous serial communication interface used in our design to drive and fill the register file with the required data and commands to be transmitted. It represents the interface with the upper control layer.

B. ***Register file:*** The memory element that stores the payload data to be transmitted, the bit rate value to be selected, the packet type (short packet or long packet) and the transmission start signal that indicates the start of the transmission operation. It also stores the coming-back done signal that indicates the end of the transmission operation.

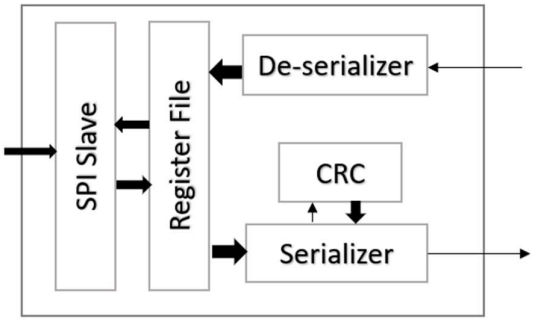

Fig. 2. Proposed Design Structure

C. ***Serializer****:* The main block of the design. It takes the data from the register file in form of bytes, then formulates the packet to be transmitted according to the packet-type signal coming from the register file. It calculates the Error-Correcting Code (ECC) on the first three bytes of the packet and appends it to the packet. For long packets, it communicates with the Cyclic Redundancy Check (CRC) block to calculate the checksum on the payload to be transmitted. Finally, it mimics the analog serial lanes that send the data serially with the required data rate specified from the register file when the transmission start signal is asserted.

D. ***Cyclic Redundancy Check (CRC):*** A 16-bit checksum calculated over the payload portion of the long data packet to immune the data payload against introduced errors.

E. ***De-serializer:*** On the receiver side, the serialized data are de-serialized again to restore back the data in the form of packets. It also generates error flags based on the errors in the received data packets.

III. PROPOSED VERIFICATION PLAN

Nowadays Integrated Circuits (IC) main challenge is to get the right design, working as intended, at the right time [4]. Verification and testing consume most of the time dedicated to delivering Intellectual Properties (IPs). Universal Verification Methodology (UVM) is proposed as a standardized verification method for verifying integrated circuit designs. It was established by Accellera in 2009, mainly derived from Open Verification Methodology (OVM), and is supported by various EDA vendors such as Cadence, Mentor Graphics, Aldec, and Synopsys [7, 8]. It introduces a standardized highly portable open-source library that enables faster development and reuse of verification environments and VIPs throughout the industry. Its class library provides highly flexible, reusable, scalable, and generic utilities. Its infrastructure includes specific-role verification components, configuration databases, testcase sequences, and data automation features. UVM is a very powerful tool to verify many of the modern standardized IPs [3, 13]. ***Our proposed verification plan is based on a constrained-random coverage driven verification technique.*** We *drive* the implemented DSI layer with *random* stimulus to *cover* various testing scenarios. Also we verify the correctness of the received data against the transmitted ones. Error Correction Code (ECC) and checksum are performed on the received packet and compared to the ECC and checksum fields in the received data. We are also defining an ***injection error mechanism*** to inject errors on the transmitted packets. Eventually, ***we are using System Verilog Assertion (SVA) to detect the injected errors on the receiver side.*** Our proposed plan presents a real test case scenario where we have a transmitter/receiver bi-directional system. On the transmitter side, we have the host processor that sends its commands, requests, and payload data serially to the receiver side. On the receiver side, we have the peripheral device that receives the data serially, converts them back to packet format, checks if errors exist, and replies with acknowledgment/error packet to the host processor.

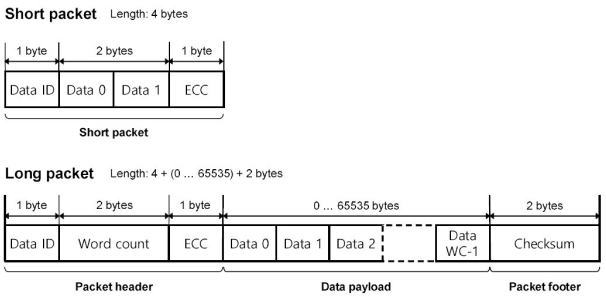

Fig. 3. long and short packet structures [2].

A. *Real-life Testcase Scenarios*

According to the DSI standard, the communication between the host processor and the peripheral device, such as a display screen, can be either half-duplex or full-duplex. In the proposed verification environment, we are classifying the test scenarios into two categories:

1) From the processor to the peripheral: DSI specification [2] defines a set of transactions from the processor to the peripheral device. Each transaction has a detailed format and unique data type. Examples of packets sent from the processor to the peripheral are the sync events (Hsync and Vsync), turn-on peripheral commands, shutdown commands and generic read/write commands.

***2) From the peripheral to the processor*:** In case of full duplex communication, the peripheral can reply to the processor with a set of defined types of packets such as: "*acknowledge packet*" when the peripheral receives data with no errors, "*acknowledge with error report packet*" indicating receiving data with errors detected, and *"response to read/ request packet"* which returns the data requested by the preceding received read command from the host processor.

The main goal is to cover all the possible real-life scenarios between the host processor as a transmitter and the peripheral device as a receiver. To reach that, short and long packets are generated with all possible configurations of their variable fields according to the standard [2]. As shown in Fig. 3, the short packet is a four bytes packet. It starts with a Data Identifier (DI) byte. The first six least significant bits of the byte define the data type to be sent while the two most significant bits identify one of four virtual channels in case of using multiple displays. The second and third bytes are data bytes. The short packet's last byte is an ECC byte, calculated over the previous three bytes.

The long packet has the same DI field as the short packet. The second field in the packet is a two-byte Word Count (WC)

that identifies the number of payload words. The third field is an ECC byte calculated over the DI and WC fields. ECC field is followed by a variable length payload of WC size. The last field is the checksum that is being calculated over the payload data. In our test plan, for both short and long packets, all fields are constrained according to the standard [2] to cover all the possible scenarios. In case of no header available or corrupted packet, the receiver will assert a set of error flags to indicate receiving of corrupted data and reply back with an error report packet in case of full-duplex communication.

B. *Error Injection Mechanism and Error-checking Information Validation*

In the proposed verification environment, we use an error injection mechanism to inject errors in the transmitted packet. This gives us the ability to test the various packet fields according to the standard specification [2] such as the ECC field, checksum field, and data ID field. On the other hand, we apply a Hamming Code ECC algorithm [11] on the received packet header and compare the results to the ECC part of the received packet. According to Hamming code theory, we need (P) parity bits to protect (D) message data bits resulting in (C) coded words, as shown in equation (1):

$$D + P + 1 <= 2^P \text{ and } C = D + P \qquad (1)$$

DSI systems use six parity bits to protect the 24-bit packet header. The most significant bits of the ECC byte (P6 and P7) are set to zeros.

In the case of long packets, we perform a standardized checksum algorithm, as shown in Fig. 4. The checksum algorithm is a 16-bit CRC with a generator polynomial represented by the equation in (2):

$$x^16 + x^12 + x^5 + x^0 \qquad (2)$$

C. *SVA Error Detection Mechanism*

In the proposed environment we use System Verilog Assertions (SVA) to detect the injected errors on the receiver side. We use system Verilog properties to check on the various possible error injection scenarios.

IV. Proposed Verification Environment Architecture

Fig. 5 shows our UVM environment architecture. It could be described as follows:

A. ***Top:*** In the environment top, we define two instances of the DUT. The tx_dut and the rx_dut. The serialized output from the tx_dut is connected to the input to the de-serializer block in the rx_dut and vice versa. We also instantiate two virtual interfaces and generate the system clocks inside our top.

B. ***Virtual interfaces:*** We have two virtual interfaces which enable the drivers to drive data to the SPI and the monitors to sample the output data. It is also the place where we define our SVA properties.

C. ***Environment:*** The proposed environment instantiates two agents, the host processor agent, and the peripheral device agent. It also instantiates a scoreboard to check data received against the transmitted ones.

D. ***Agents:*** In the case of full duplex communication, we have two active agents that can drive data sequences to the DUT. In the case of half duplex communication, the peripheral agent acts as a passive agent where it will not be able to drive sequences, even though it will always be able to monitor the interface signals.

E. ***Transactions, sequences, and sequencers:*** Transactions contain the data items that are randomized in sequences and sent to the drivers through their sequencers. Data items are randomized to cover all the possible real-life scenarios according to the standard specification [2]. Faulty sequence items are added to be used in case of error injection scenarios using directed testcases.

F. ***Virtual sequencer and virtual sequence lib:*** Virtual sequencer is the central place where we can control the sequencers of the two available agents. It contains handles for these sequencers. The virtual sequence lib is the place where we define the sequence classes to drive the DUT.

G. ***Driver:*** The driver gets the transactions from the sequencer and starts driving the SPI interface through the virtual interface. It acts as the upper control layer of the DUT.

H. ***Monitors:*** The monitor tracks the interface signals, samples the important information, and sends them to the scoreboard through the transactions. It is also the place where we define our Functional coverage

I. ***Scoreboard***: It collects the transactions from both agents. Then checks the packets at one side against the expected packets, based on the collected information from the other side and the ECC and checksum calculations.

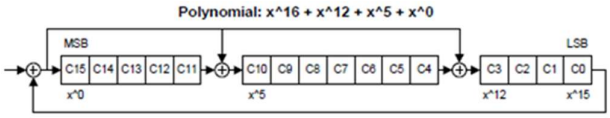

Fig. 4. 16-bit CRC Generation Using a Shift Register [2]

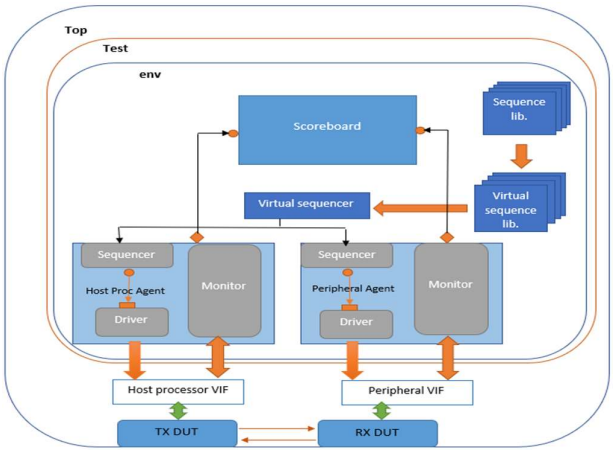

Fig. 5. Proposed Environment Architecture

V. Results

A summary of the obtained RTL simulation results can be described in the following points:

A. RTL simulation results and covering real-life scenarios: Our RTL implementation of the DSI low-level protocol layer can perform half/full duplex communication operations according to the DSI standard specification [2]. In the proposed environment, we can cover all the possible real-

life scenarios between a host protocol and a peripheral device according to the specifications [2, 9]. We used a set of test case scenarios that randomize each packet field within acceptable standardized values. Fig. 6 shows a testcase scenario in which the host protocol sends a long packet of a generic data type (data identifier field = 29 according to the standard [2]), followed by an ack short packet (data identifier = 2 with null data words) sent back from the peripheral device to the host processor.

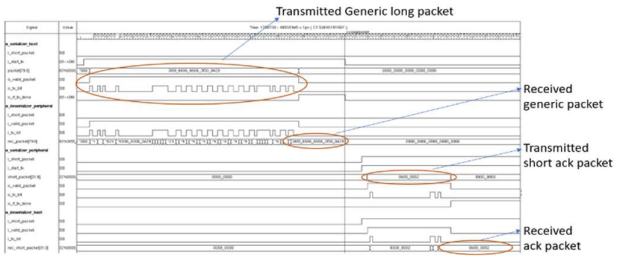

Fig. 6. Generic long packet followed by ack short packet testcase scenario

B. Error injection mechanism: An error injection methodology was implemented in which we inject faulty UVM sequence items using directed UVM sequences on the transmission side and detect these errors using SVA on the receiver side. We were able to detect all the possible error flags *supported by the low-level protocol layer* according to the standard [2]. Table. 1 summarizes these errors and their description.

TABLE 1. Error injection detection summary

Error description	Injection methodology	Detection Methodology	Results in RTL simulations
ECC Error, single bit (detected and corrected)	Faulty UVM transaction	SVA	Detected successfully
ECC Error, multi-bit (detected, not corrected)	Faulty UVM transaction	SVA	Detected successfully
Checksum Error (Long packet only)	Faulty UVM transaction	SVA	Detected successfully
DSI Data Type Not Recognized	Faulty UVM transaction	SVA	Detected successfully
DSI VC ID Invalid	Faulty UVM transaction	SVA	Detected successfully
Invalid Transmission Length	Faulty UVM transaction	SVA	Detected successfully

C. SVA Error detection mechanism: An example of an *invalid data-identifier (DSI data type not recognized)* error injection and its correct detection using SVA is presented in Fig. 7.

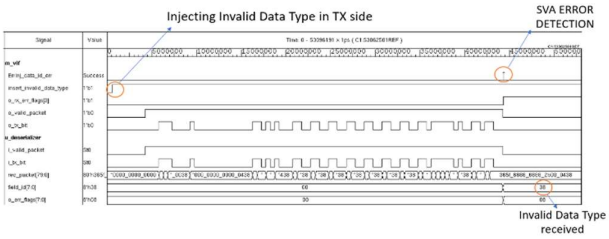

Fig. 7. Error injection of invalid data identifier

D. Code and Functional coverage: We were able to reach the required full functional coverage, indicating that we can hit all the required real-life scenarios. We also reached a 93% code coverage of the design under test. Coverage results can be shown in Fig. 8.

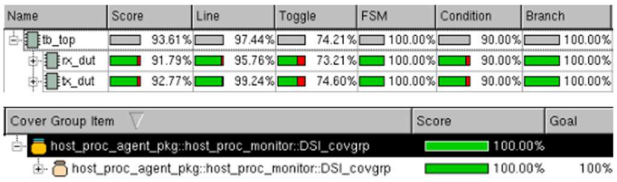

Fig. 8. Code and functional coverage results

VI. CONCLUSION

This paper proposed a UVM constrained random coverage-driven verification approach for DSI low-level protocol layer. This approach could be a basis for a standardized, scalable, and reusable Verification Intellectual Property (VIP) to test and verify the standardized protocol layer. Various verification techniques were used such as an error injection mechanism using injected faulty sequence items and an error detection mechanism using system Verilog assertions. We were able to cover the different real-life scenarios between the host processor and the peripheral device according to the standard specification [2], reaching 100% functional coverage and 93% code coverage.

REFERENCES

[1] http://www.mipi.org/

[2] MIPI Alliance, MIPI alliance specification for display serial interface, version 1.3.2, 2020.

[3] El-Naggar, A., Massoud, E., Medhat, A., Ibrahim, H., Al-Abassy, B., El-Ashry, S., Khamis, M. and Shalaby, A., 2016, December. A narrative of UVM testbench environment for interconnection routers: A practical approach. In 2016 11th International Design & Test Symposium (IDT) (pp. 98-103). IEEE.

[4] Bergeron, J., (2006). Writing Testbenche using SystemVerilog (3rd ed). New York, NY: Springer.

[5] Pandey, A. K., Jangale, A., & Narayan, S. (2020, May). Signal Integrity and Compliance Test of DSI and CSI2 Serial Interface over MIPI D-PHY. In 2020 IEEE 24th Workshop on Signal and Power Integrity (SPI) (pp. 1-4). IEEE.

[6] Rodrigues, P. F. X. (2016). MIPI Display Serial Interface Interoperability Tests (Doctoral dissertation, Instituto Politecnico do Porto (Portugal)).

[7] IEEE Standard for Universal Verification Methodology Language Reference Manual: https://ieeexplore.ieee.org/document/7932212/

[8] https://verificationacademy.com/cookbook/uvm

[9] MIPI Alliance, MIPI alliance specification for display serial interface, version 1.3.2 diff v1.3.1, 2020.

[10] https://www.mipi.org/specifications/dsi

[11] Rahim, R. (2017). Bit error detection and correction with hamming code algorithm. Int. J. Sci. Res. Sci. Eng. Technol., vol. 3, pp. 76-81

[12] Kim, Y. J., Kim, D. H., Kim, S. M., & Cho, K. R. (2010). Low Power Design of a MIPI Digital D-PHY for the Mobile Signal Interface. The Journal of the Korea Contents Association, 10(12), 10-17.

[13] Melikyan, V., Harutyunyan, S., Kirakosyan, A., & Kaplanyan, T. (2021, September). UVM Verification IP for AXI. In 2021 IEEE East-West Design & Test Symposium (EWDTS) (pp. 1-4). IEEE.